

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



SOICT

BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG LƯU TRỮ, XỬ LÝ, PHÂN TÍCH VÀ DỰ ĐOÁN DỮ LIỆU MẠNG XÃ HỘI

Thành viên	MSSV
Nguyễn Kiêm Khang	20235115
Lê Kiên Cường	20235025
Trần Tiến Dũng	20235056
Võ Hữu Trí Dũng	20235057
Nguyễn Hữu Dũng	20235052

Giảng viên hướng dẫn: TS. Trần Việt Trung

Hà Nội, tháng 6 năm 2026

Lời mở đầu

Đồ án này nghiên cứu và triển khai hệ thống Big Data phân tích dữ liệu mạng xã hội đa nền tảng dựa trên **Kiến trúc Lambda**. Hệ thống giải quyết bài toán xử lý khối lượng lớn nội dung từ ba nền tảng Reddit, Facebook và Instagram thông qua hai luồng song song: **Lớp Batch** (sử dụng Apache Airflow, PySpark, MinIO) để tổng hợp dữ liệu lịch sử và huấn luyện mô hình Machine Learning; và **Lớp Speed** (sử dụng Apache Kafka, Spark Structured Streaming) để xử lý luồng sự kiện mạng xã hội theo thời gian thực, làm giàu dữ liệu với NLP Pipeline.

Kết quả phân tích được lưu trữ tối ưu trên **MinIO** (cho dữ liệu thô và batch views), **Elasticsearch** và **Redis** (cho dữ liệu thời gian thực), và **ClickHouse** (cho data warehouse OLAP), đồng thời được trực quan hóa trên Dashboard. Hệ thống không chỉ cung cấp các chỉ số phân tích vĩ mô về xu hướng cảm xúc (Sentiment Trend), hashtag nổi bật, phân tích cộng đồng (Community Detection) mà còn có khả năng tự động cập nhật và dự đoán khả năng lan truyền (Virality Prediction) của bài đăng bằng mô hình LightGBM kết hợp PhoBERT embeddings. Mọi luồng hoạt động đều được đảm bảo độ tin cậy bằng hệ thống kiểm thử toàn diện.

Chúng em xin chân thành cảm ơn thầy Trần Việt Trung đã trang bị cho chúng em những kiến thức nền tảng vững chắc trong suốt quá trình học tập và rèn luyện tại trường.

Mặc dù đã nỗ lực hết mình, đồ án chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp ý kiến từ thầy để đề tài được hoàn thiện và phát triển tốt hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

Contents

Lời mở đầu	1
1 Xác định vấn đề	3
1.1 Vấn đề đã chọn	3
1.2 Phân tích sự phù hợp của đề tài với Dữ liệu lớn	3
1.3 Phạm vi và hạn chế	4
2 Kiến trúc hệ thống	4
2.1 Kiến trúc tổng quan	4
2.2 Chi tiết các thành phần và vai trò	5
2.2.1 Ingestion Layer	5
2.2.2 Batch Layer (Lớp xử lý lô)	5
2.2.3 Speed Layer (Lớp xử lý thời gian thực)	6
2.2.4 Serving Layer	7
2.3 Luồng dữ liệu	7
3 Chi tiết triển khai	8
3.1 Mã nguồn & Quản lý Môi trường	8
3.2 Các tệp cấu hình theo từng môi trường	9
3.3 Triển khai	10
3.4 Chi tiết thực thi quy trình xử lý	11
3.4.1 Xử lý lô và Tổng hợp thống kê	11
3.4.2 Xử lý Luồng Thời gian Thực	14
3.4.3 Cơ chế Tự động Huấn luyện lại mô hình (ML Retraining Pipeline)	16
3.4.4 Lưu trữ vào Elasticsearch (Realtime Sink)	17
3.4.5 Merge Service — Hợp nhất Batch và Speed View	17
3.5 Kiểm thử và Đảm bảo chất lượng (QA)	18
3.6 Thiết lập giám sát	18
3.7 Giao diện người dùng và Trực quan hóa kết quả	21
3.7.1 Phân hệ Giám sát Thời gian thực (Real-time View)	21
3.7.2 Phân hệ Phân tích Lịch sử (Historical View)	22
4 Bài học kinh nghiệm	23
4.1 Bài học 1: Spark Out-Of-Memory khi tích hợp NLP vào Batch Job	23
4.2 Bài học 2: Kubernetes Pod Crash do Resource Limits không phù hợp	25
4.3 Bài học 3: Architectural Pivot — Revert Merge Service từ ClickHouse về Elasticsearch/Redis	27
4.4 Bài học 4: NLP Pipeline Cold-Start và PhoBERT Loading Overhead trong Speed Layer	29
4.5 Bài học 5: Small Files Problem — Chiến lược Flush Kafka → MinIO	30
4.6 Bài học 6: Data Deduplication trong Serving Layer — Merge Strategy cho Lambda Architecture	32
4.7 Tổng hợp các bài học khác	35

1 Xác định vấn đề

1.1 Vấn đề đã chọn

Mạng xã hội hiện đại tạo ra một lượng dữ liệu khổng lồ mỗi giây, từ bài đăng văn bản, hình ảnh, video cho đến bình luận và phản ứng của người dùng. Theo ước tính, hàng trăm triệu bài đăng được tạo ra mỗi ngày trên các nền tảng lớn như Reddit, Facebook và Instagram. Dữ liệu này chứa đựng thông tin vô cùng giá trị về xu hướng dư luận, tâm lý cộng đồng, sức ảnh hưởng của các tác nhân (influencers) và mức độ lan truyền của nội dung. Tuy nhiên, thách thức lớn nhất là làm thế nào để thu thập, lưu trữ, xử lý và rút trích tri thức từ luồng dữ liệu này một cách kịp thời và chính xác.

Dựa trên bối cảnh đó, nhóm lựa chọn đề tài: *“Xây dựng hệ thống lưu trữ, xử lý, phân tích và dự đoán dữ liệu mạng xã hội theo Kiến trúc Lambda.”*

Hệ thống giải quyết hai nhu cầu cốt lõi:

- **Xử lý lịch sử (Batch Layer):** Tổng hợp và phân tích dữ liệu bài đăng theo thời gian để phát hiện xu hướng cảm xúc, hashtag nổi bật, mạng lưới tương tác tác giả và huấn luyện mô hình Machine Learning dự đoán độ lan truyền nội dung.
- **Xử lý thời gian thực (Speed Layer):** Tiếp nhận luồng bài đăng đang diễn ra từ nhiều nền tảng, làm giàu với NLP Pipeline (phân tích cảm xúc, trích xuất từ khóa, nhận dạng thực thể) và cập nhật Dashboard trong thời gian gần thực.

1.2 Phân tích sự phù hợp của đề tài với Dữ liệu lớn

Bài toán phân tích mạng xã hội hoàn toàn phù hợp với Big Data vì thỏa mãn các đặc trưng **5Vs**:

- **Volume (Dung lượng lớn):** Dữ liệu mạng xã hội tích lũy theo thời gian là rất lớn. Tập dữ liệu của hệ thống bao gồm dữ liệu từ ba nền tảng (Reddit CSV, Facebook JSON, Instagram JSONL) với tổng dung lượng lên tới 3.7GB, trong đó 2.3GB là dữ liệu thuần. Hệ thống sử dụng **MinIO** kết hợp với **PySpark** để cho phép xử lý song song khối lượng dữ liệu này.
- **Velocity (Tốc độ cao):** Nội dung mạng xã hội được tạo ra liên tục theo thời gian thực. Hệ thống sử dụng **Apache Kafka** làm message bus trung tâm và **Spark Structured Streaming** với micro-batch 5 giây để đảm bảo độ trễ thấp trong việc cập nhật Dashboard.
- **Variety (Đa dạng dữ liệu):** Dữ liệu đầu vào bao gồm nhiều định dạng khác nhau: CSV (Reddit), JSON (Facebook), JSONL/NDJSON (Instagram). Nội dung cũng đa dạng: văn bản, URL media, cây bình luận lồng nhau (nested comment tree), hashtag. Hệ thống cần tích hợp và chuẩn hóa tất cả nguồn này về một schema thống nhất (SOP Schema).
- **Veracity (Chất lượng dữ liệu):** Dữ liệu mạng xã hội thường chứa nhiều nhiễu: bài đăng trùng lặp (do cơ chế At-least-once của Kafka), nội dung thiếu trường bắt buộc, timestamp không hợp lệ, hoặc nội dung rỗng. Hệ thống triển khai pipeline làm sạch dữ liệu bài bản với validation schema, deduplication theo `post_id`, và Dead Letter Queue (DLQ) cho các bản ghi lỗi. Sự tin cậy còn được đảm bảo bằng bộ test toàn diện (E2E, Load Test).

- **Value (Giá trị khai thác):** Dữ liệu thô từ ba nền tảng chỉ trở thành tài sản khi được chuyển hóa thành thông tin hữu ích. Hệ thống tạo ra giá trị cụ thể: phân tích xu hướng sentiment theo giờ/ngày/tuần, xếp hạng top hashtag hàng tuần, phát hiện cộng đồng qua **Network Analysis** (PageRank + Louvain Community Detection), xác định tác giả có ảnh hưởng (author activity), và cung cấp toàn bộ kết quả qua **FastAPI REST API** và Dashboard trực quan hóa — cho phép ra quyết định dựa trên dữ liệu trong lĩnh vực truyền thông xã hội và marketing.

1.3 Phạm vi và hạn chế

Phạm vi: Xây dựng pipeline End-to-End gồm Ingestion (Kafka Simulator), Batch (Airflow, PySpark, MinIO, ClickHouse), Speed (Spark Structured Streaming, Redis, Elasticsearch), MLOps (Retraining DAG) và Serving (FastAPI, Dashboard).

Hạn chế:

- Sử dụng dữ liệu giả lập/ lịch sử thay vì API thực tế của các nền tảng mạng xã hội (do giới hạn về quyền truy cập API và chi phí).
- Hệ thống được triển khai trên cụm Kubernetes cục bộ (Minikube) thay vì hệ thống phân tán vật lý đa node. Do đó, hiệu năng và khả năng chịu lỗi bị giới hạn bởi tài nguyên của máy trạm (tối thiểu 10GB RAM, 4 CPU), chưa phản ánh đúng quy mô Production.
- Mô hình ML (PhoBERT fine-tuned, LightGBM) tập trung vào quy trình kỹ thuật dữ liệu (Data Engineering & MLOps) hơn là tối ưu hóa độ chính xác thuật toán. Nhân pseudo-label được tạo tự động bằng lexicon thay vì nhân con người.
- Phân tích mạng (Network Analysis) với PageRank và Louvain Community Detection sử dụng NetworkX, phù hợp cho quy mô vừa nhưng chưa có khả năng scale với đồ thị hàng triệu node.

2 Kiến trúc hệ thống

2.1 Kiến trúc tổng quan

Nhóm áp dụng mô hình **Lambda Architecture** để cân bằng giữa độ chính xác cao (Batch Layer) và độ trễ thấp (Speed Layer) [1]. Đây là kiến trúc chuẩn được sử dụng rộng rãi trong các hệ thống Big Data hiện đại vì khả năng xử lý cả hai loại truy vấn: truy vấn phân tích sâu trên dữ liệu lịch sử và truy vấn cập nhật theo thời gian thực.

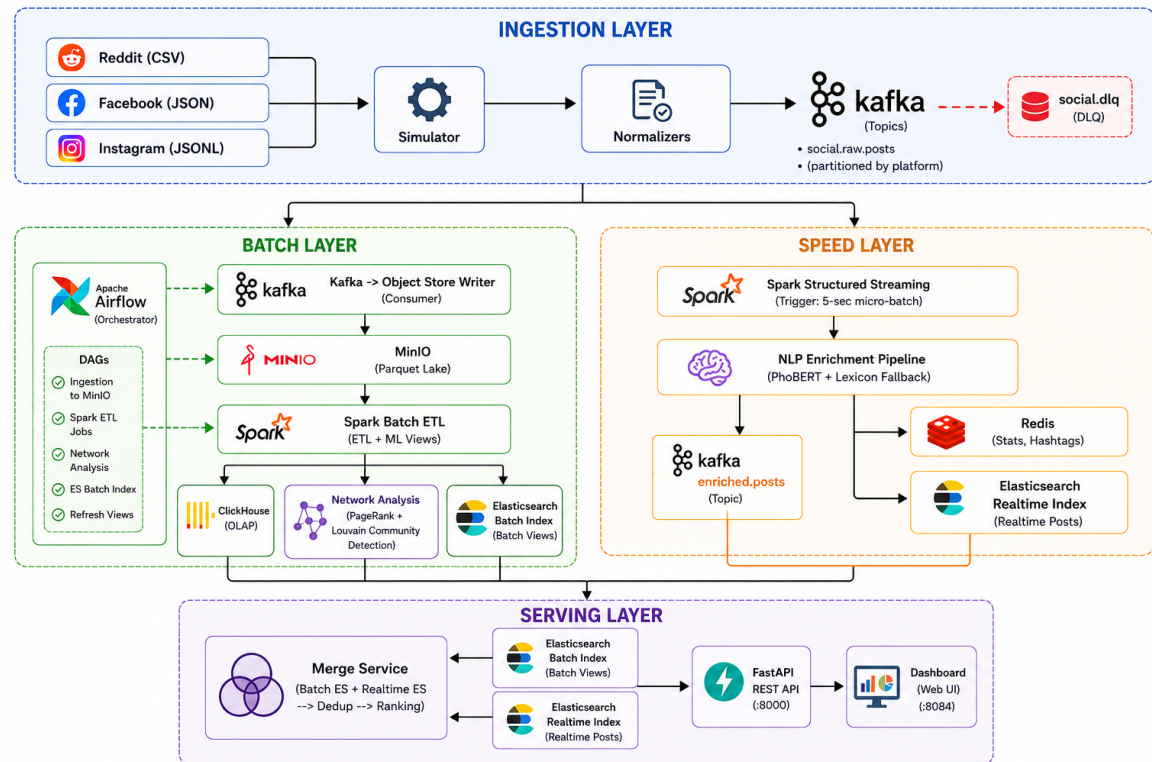


Figure 1: Sơ đồ kiến trúc hệ thống phân tích dữ liệu mạng xã hội (Lambda Architecture)

2.2 Chi tiết các thành phần và vai trò

2.2.1 Ingestion Layer

- **Apache Kafka (KRaft mode):** Message Queue trung tâm, nhận dữ liệu từ các Simulator và phân phối cho Batch Layer và Speed Layer. Kafka được cấu hình với 5 topic chính: `social.reddit.posts`, `social.facebook.posts`, `social.instagram.posts`, `social.enriched.posts` (sau NLP enrichment), và `social.dlq` (Dead Letter Queue cho bản ghi lỗi). [2]
- **Platform Simulators:** Ba Kubernetes Job độc lập (Reddit, Facebook, Instagram) replay dữ liệu từ file tĩnh (CSV/JSON/JSONL) vào Kafka với tốc độ có kiểm soát. Mỗi simulator thực hiện chuẩn hóa dữ liệu đặc thù theo nền tảng trước khi gửi lên Kafka.
- **Normalizers:** Module Python riêng cho từng nền tảng (`ingestion/normalizers/`), chuyển đổi schema nguồn (schema Reddit, Facebook, Instagram khác nhau hoàn toàn) về một schema chuẩn thống nhất (SOP Schema) với các trường bắt buộc: `post_id`, `platform`, `author_id`, `content`, `hashtags`, `created_at`, `metrics`.

2.2.2 Batch Layer (Lớp xử lý lô)

Nơi lưu trữ toàn bộ dữ liệu thô và thực hiện các tính toán nặng định kỳ.

- **MinIO:** Object Storage S3-compatible, đóng vai trò là **Data Lake** của hệ thống. Lưu trữ toàn bộ dữ liệu thô (Raw Data) dưới định dạng Parquet, phân vùng theo

platform/year/month/day. MinIO cũng lưu trữ các Batch Views (kết quả tính toán của Spark) và Checkpoints của Airflow DAG. [6]

- **Apache Airflow:** Công cụ điều phối (Orchestrator), lên lịch và quản lý luồng thực thi các Job theo DAG. DAG `social_lambda_batch_pipeline` được kích hoạt mỗi 5 phút, kiểm tra dữ liệu mới trong MinIO và thực thi tuần tự các bước: Object Store Write → Spark ETL → ClickHouse Load → Elasticsearch Index → Network Analysis. Ngoài ra, Airflow còn quản lý DAG `virality_retrain_dag` cho tác vụ MLOps.
- **PySpark (Batch Processing):** Đọc dữ liệu Parquet từ MinIO để: [3]
 1. Làm sạch và tổng hợp dữ liệu thống kê (ETL): tạo các Batch Views như `platform_daily_stats`, `top_hashtags_weekly`, `author_activity`, `sentiment_time_series`, `top_posts`.
 2. Huấn luyện và áp dụng mô hình Machine Learning: Sentiment Analysis (PhoBERT fine-tuned + Lexicon fallback) và Virality Prediction (LightGBM).
- **ClickHouse:** OLAP Data Warehouse, nhận kết quả từ Spark Batch Job. Sử dụng engine `ReplacingMergeTree` để hỗ trợ cập nhật idempotent. (*Lưu ý: ClickHouse phục vụ cho các truy vấn phân tích sâu (Analytics/Dashboard), trong khi các truy vấn API trực tiếp yêu cầu độ trễ thấp được phục vụ bởi Elasticsearch - xem chi tiết tại Bài học 3*).

2.2.3 Speed Layer (Lớp xử lý thời gian thực)

- **Spark Structured Streaming:** Subscribe Kafka topics để xử lý luồng bài đăng theo cơ chế micro-batch với trigger 5 giây. Thực hiện parse JSON, validate schema, lọc bản ghi ngoài khoảng thời gian hợp lệ (2026-01-01 đến 2026-04-30), và chuyển các bản ghi lỗi sang DLQ.
- **NLP Enrichment Pipeline:** Sau khi nhận được batch từ Spark Streaming, mỗi bài đăng được làm giàu với: [4]
 - **Phân tích cảm xúc (Sentiment Analysis):** Sử dụng mô hình PhoBERT fine-tuned (hỗ trợ tiếng Việt) nếu có sẵn, ngược lại fallback sang phân tích từ điển (Lexicon-based) với hỗ trợ emoji detection.
 - **Trích xuất từ khóa (Keyword Extraction):** Top-N noun chunks qua spaCy.
 - **Nhận dạng thực thể (NER):** Phát hiện tổ chức, người, địa điểm, sản phẩm.
 - **Phát hiện ngôn ngữ:** Sử dụng thư viện `langdetect`.
- **Redis:** Lưu trữ thống kê thời gian thực dưới dạng key-value với TTL ngắn (2 giờ cho stats, 1 giờ cho hashtags). Sử dụng Sorted Set cho top hashtags, cho phép truy vấn ZRANGE với độ phức tạp $O(\log N)$.
- **Elasticsearch (Realtime Index):** Lập chỉ mục toàn bộ bài đăng đã enriched vào index `social_realtime_views`, hỗ trợ full-text search và date-range filter. Cập nhật gần thực (mỗi 5 giây theo trigger của Spark Streaming).

2.2.4 Serving Layer

Hợp nhất kết quả từ hai lớp trên để cung cấp dữ liệu nhất quán cho ứng dụng cuối (Dashboard và API). Lớp Serving áp dụng mô hình phân tách trách nhiệm (Separation of Concerns) với các service độc lập:

- **Elasticsearch (Batch Views):** Index `social_batch_views` chứa kết quả tổng hợp lịch sử từ Spark Batch Job. Phục vụ các truy vấn phân tích sâu không yêu cầu thời gian thực.
- **Merge Service (ServeQuery):** Thành phần quan trọng nhất cho bài đăng. Hợp nhất (merge) kết quả từ Elasticsearch Batch Index và Elasticsearch Realtime Index bằng cách deduplication theo `post_id`, giữ lại bản ghi mới nhất theo `event_ts`. Đây là cơ chế cốt lõi của Lambda Architecture để đảm bảo tính nhất quán dữ liệu.
- **Network Service & Topic Service:** Phục vụ truy vấn cho đồ thị tương tác Node-Edge và phân phối các chủ đề (Topic Modeling).
- **FastAPI REST Server:** Cung cấp 15+ endpoints REST API tại cổng 8000, bao gồm truy vấn bài đăng, xu hướng cảm xúc, hashtag, thống kê realtime, phân tích chủ đề, mạng lưới tác giả, và quản lý mô hình ML.
- **Dashboard:** Giao diện web tĩnh tại cổng 8084, trực quan hóa kết quả phân tích theo thời gian thực và lịch sử.

2.3 Luồng dữ liệu

Hệ thống vận hành theo hai luồng song song:

1. Cold Path (Batch Processing):

- **B1:** Simulator đọc file dữ liệu thô (CSV/JSON/JSONL), normalize và đẩy vào Kafka topics theo tốc độ giới hạn (20 records/giây, có token bucket rate limiter).
- **B2:** `object-store-writer` là Kafka Consumer liên tục đọc từ 3 source topics, tích lũy trong buffer và flush định kỳ (500 bản ghi hoặc 60 giây) xuống MinIO dưới dạng Parquet phân vùng theo ngày.
- **B3:** Airflow kích hoạt Spark Batch Job. PySpark đọc Parquet từ MinIO, thực hiện ETL, tính toán Batch Views và ghi kết quả trở lại MinIO. Tại đây quy trình rẽ nhánh:
 - *Nhánh Analytics:* Tính toán các chỉ số KPI và ghi vào ClickHouse (Batch View). Dữ liệu này sẽ được hiển thị trên Dashboard Historical tab.
 - *Nhánh ML:* Thực hiện Feature Engineering và huấn luyện mô hình Virality Prediction, lưu artifact mô hình (cơ chế Safe Artifacts đảm bảo tính an toàn).
- **B4:** `elasticsearch-indexer` đọc Batch Views từ MinIO và index vào `social_batch_views` Elasticsearch index.
- **B5:** `network-analysis` xây dựng đồ thị tương tác tác giả, tính PageRank và Louvain communities, ghi kết quả vào Elasticsearch và Redis.

2. Hot Path (Real-time Processing):

- **H1:** Bài đăng mới được đẩy vào Kafka (cùng luồng với Cold Path, nhưng được tiêu thụ bởi Speed Layer Consumer).
- **H2:** Spark Structured Streaming tiêu thụ dữ liệu từ Kafka ngay lập tức theo micro-batch 5 giây.
- **H3:** Mỗi bài đăng được làm giàu bằng NLP Pipeline (sentiment, keywords, NER, language detection) trong hàm `foreach_batch`.
- **H4:** Kết quả được ghi đồng thời vào: Redis (stats có TTL), Elasticsearch `social_realtime_views` (searchable index), và Kafka topic `social.enriched.posts` (cho downstream consumers).

3 Chi tiết triển khai

3.1 Mã nguồn & Quản lý Môi trường

Hệ thống sử dụng công cụ quản lý package hiện đại **uv** (viết bằng Rust) thay vì **pip** truyền thống. Cấu hình dependencies được định nghĩa trong `pyproject.toml` và chốt phiên bản tại `uv.lock`, giúp tăng tốc độ build Docker image và đảm bảo tính nhất quán qua các môi trường.

- **GitHub Repository:** <https://github.com/vohuutridung/IT4931>
- **Cấu trúc thư mục:**

```

1 IT4931/
2 |-- k8s/                # Kubernetes manifests (00-09, theo thu tu
   phu thuoc)
3 |   |-- 01-config/      # ConfigMap, Secrets
4 |   |-- 02-storage/    # PersistentVolumeClaims
5 |   |-- 03-infrastructure/ # Kafka, MinIO, Redis, PostgreSQL,
   ClickHouse, ES
6 |   |-- 04-apps/        # Spark Master/Worker, API, Dashboard,
   Streaming
7 |   |-- 05-orchestration/ # Airflow (webserver + scheduler)
8 |   '-- 07-simulators/   # Reddit, Facebook, Instagram replay
   jobs
9 |-- ingestion/          # Simulator va Normalizers
10 | |-- simulator.py      # File replay voi token bucket rate limiter
11 | '-- normalizers/     # reddit.py, facebook.py, instagram.py
12 |-- batch/              # Batch Layer
13 | |-- object_store_writer.py # Kafka -> MinIO Parquet writer
14 | |-- spark_batch_job.py  # ETL, Batch Views, Sentiment
15 | '-- network_analysis.py # PageRank + Louvain Communities
16 |-- warehouse/          # Data Warehouse Layer
17 | '-- clickhouse_loader.py # Load Batch Views vao ClickHouse
18 |-- speed/              # Speed Layer
19 | |-- streaming_job.py   # Spark Structured Streaming entrypoint
20 | |-- nlp_pipeline.py   # PhoBERT + Lexicon NLP enrichment
21 | '-- realtime_stores.py # Ghi vao Redis va Elasticsearch
22 |-- serving/            # Serving Layer
23 | |-- merge_service.py   # ServeQuery: merge Batch + Realtime
   views
24 | |-- network_service.py # Phục vụ do thi tuong tac Node-Edge
25 | |-- topic_service.py   # Topic modeling queries

```

```

26 | |-- es_indexer.py          # Elasticsearch index management
27 | |-- api.py               # FastAPI REST endpoints
28 | |-- ml/                  # Machine Learning components
29 | |-- sentiment/          # PhoBERT fine-tuning pipeline
30 | |-- virality/           # LightGBM + PhoBERT virality predictor
31 | |-- orchestration/
32 | |-- dags/batch_pipeline_dag.py # Airflow Batch DAG
33 | |-- dags/virality_retrain_dag.py # Airflow Retrain DAG
34 | |-- tests/              # Hệ thống kiểm thử (QA)
35 | |-- e2e/               # Test toàn luồng (pipeline contract)
36 | |-- integration/       # Test tích hợp
37 | |-- load/              # Load testing với Locust
38 | |-- unit/              # Unit test cho từng module
39 | |-- uv.lock             # Quản lý dependency với uv
40 | |-- pyproject.toml
41 | |-- docker/             # Dockerfiles (social-python, social-spark,
    | airflow)

```

Listing 1: Cấu trúc dự án IT4931

3.2 Các tệp cấu hình theo từng môi trường

Hệ thống áp dụng nguyên tắc **Externalized Configuration** (Tách biệt cấu hình khỏi code). Toàn bộ biến môi trường và thông tin nhạy cảm được quản lý thông qua Kubernetes ConfigMaps và Secrets:

- **Namespace:** Toàn bộ tài nguyên được cô lập trong namespace `social-pipeline` (`k8s/00-namespace.yaml`).
- **ConfigMaps:** File `k8s/01-config/configmap.yaml` chứa các cấu hình chung như địa chỉ Kafka Bootstrap Server, MinIO Endpoint, Spark Master URL, và các tham số pipeline như `CONSUMER_FLUSH_SIZE=500`, `STREAM_TRIGGER_SECS=5`, `REALTIME_WINDOW_HOURS=24`.
- **Secrets:** Thông tin đăng nhập (MinIO credentials, ClickHouse password, Elasticsearch auth) được khởi tạo an toàn thông qua Kubernetes Secret, không hard-code trong mã nguồn:

```

1 kubectl create secret generic social-pipeline-secrets \
2   --from-literal=S3_ACCESS_KEY=minioadmin \
3   --from-literal=S3_SECRET_KEY=minioadmin \
4   --from-literal=CLICKHOUSE_PASSWORD=social \
5   -n social-pipeline

```

Listing 2: Tạo Kubernetes Secret cho hệ thống

- **Central Settings (config/settings.py):** Module Python đọc toàn bộ cấu hình từ biến môi trường với giá trị mặc định cho môi trường dev. Đặc biệt, có cơ chế bảo vệ Production:

```

1 # config/settings.py
2 KAFKA_BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "
    | localhost:9092")
3 S3_BUCKET = os.getenv("S3_BUCKET", "social-lake")
4 STREAM_TRIGGER_SECS = int(os.getenv("STREAM_TRIGGER_SECS", "5"))
5

```

```
6 # Bao ve tranh dung credential mac dinh tren Production
7 if ENV == "production" and "minioadmin" in S3_ACCESS_KEY:
8     raise ValueError("ERROR: Using default S3 credentials in
      production!")
```

Listing 3: Cấu hình trung tâm với bảo vệ Production

3.3 Triển khai

Chiến lược triển khai tuân theo mô hình **On-premise Kubernetes Simulation** sử dụng Minikube. Quy trình được chia thành 4 giai đoạn chính:

Giai đoạn 1: Khởi tạo môi trường

Khởi tạo Cluster với tài nguyên được tính toán kỹ lưỡng để chịu tải được Spark, Kafka, và NLP models:

```
1 # Cap phat 10GB RAM va 4 CPU cho Minikube voi mount thu muc data
2 minikube start --memory=10240 --cpus=4 \
3     --mount --mount-string="$(pwd):/social-pipeline"
4
5 # Yeu cau bat buoc cua Elasticsearch
6 minikube ssh -- sudo sysctl -w vm.max_map_count=262144
```

Listing 4: Khởi tạo Minikube cluster

Giai đoạn 2: Triển khai tầng hạ tầng

Triển khai các Stateful Services theo thứ tự phụ thuộc (Bao gồm cả PostgreSQL đóng vai trò làm **Metadata Database cho Apache Airflow**, giúp lưu trữ trạng thái các DAGs/Tasks):

```
1 kubectl apply -f k8s/01-config/           # ConfigMap va Secrets
2 kubectl apply -f k8s/02-storage/         # PersistentVolumeClaims (10
      GB/platform)
3 kubectl apply -f k8s/03-infrastructure/   # Kafka, MinIO, Redis,
      PostgreSQL, CH, ES
```

Listing 5: Triển khai hạ tầng Kubernetes

Lưu ý: Sau bước này, thực hiện Port-forward MinIO (port 9001) để tạo bucket social-lake.

Giai đoạn 3: Xây dựng và phân phối ứng dụng

Sử dụng Docker daemon của Minikube để build image trực tiếp, tránh việc phải push/pull từ Docker Hub:

```
1 # Ket noi Docker CLI voi Minikube daemon
2 eval $(minikube docker-env)
3
4 # Build cac image theo thu tu phu thuoc (quan ly dependency qua uv)
5 docker build -t social-python:0.1.0 -f docker/Dockerfile.python .
6 docker build -t social-spark:3.5.1 -f docker/Dockerfile.spark .
7 docker build -t social-pipeline-airflow -f docker/Dockerfile.
      airflow .
```

Listing 6: Build Docker images với Minikube daemon

Giai đoạn 4: Triển khai các thành phần xử lý và điều phối

```

1 # 1. Trien khai Speed Layer (luon kich hoat truoc)
2 kubectl apply -f k8s/04-apps/speed-streaming.yaml
3 kubectl apply -f k8s/04-apps/object-store-writer.yaml
4 kubectl apply -f k8s/04-apps/api.yaml
5 kubectl apply -f k8s/04-apps/dashboard.yaml
6
7 # 2. Trien khai Orchestrator (Batch Layer)
8 kubectl apply -f k8s/05-orchestration/
9
10 # 3. Kich hoat Simulators de bat dau replay du lieu
11 kubectl apply -f k8s/07-simulators/
12
13 # 4. Port-forward de truy cap dich vu
14 kubectl port-forward svc/api 8000:8000 -n social-pipeline &
15 kubectl port-forward svc/dashboard 8084:8084 -n social-pipeline &
16 kubectl port-forward svc/airflow-webserver 8085:8080 -n social-
    pipeline &

```

Listing 7: Triển khai ứng dụng và điều phối

Các dịch vụ sau khi triển khai được truy cập tại:

Table 1: Danh sách dịch vụ sau khi triển khai

Dịch vụ	URL	Tài khoản
Dashboard	http://localhost:8084	—
REST API	http://localhost:8000	—
Airflow UI	http://localhost:8085	admin/admin
MinIO Console	http://localhost:9001	minioadmin/minioadmin
Spark UI	http://localhost:8080	—
ClickHouse	http://localhost:8123	social/social
Elasticsearch	http://localhost:9200	—

3.4 Chi tiết thực thi quy trình xử lý

Trong phần này, nhóm đi sâu vào logic xử lý dữ liệu tại hai lớp Batch và Speed, bao gồm các công thức tổng hợp và quy trình áp dụng mô hình máy học.

3.4.1 Xử lý lô và Tổng hợp thống kê

3.4.1.a Thu thập dữ liệu vào Data Lake Đây là bước khởi đầu của Cold Path. `object-store-writer` là một Kafka Consumer liên tục chạy trong Kubernetes, có nhiệm vụ đọc dữ liệu thô từ Kafka và lưu trữ bền vững vào MinIO dưới định dạng Parquet.

Thách thức chính là làm thế nào để cân bằng giữa **độ trễ** (flush nhanh = nhiều file nhỏ) và **hiệu quả I/O** (flush chậm = ít file lớn hơn). Nhóm áp dụng chiến lược **Dual-trigger Flush**: flush khi đạt 500 bản ghi HOẶC sau 60 giây, tùy điều kiện nào đến trước:

```

1 # batch/object_store_writer.py

```

```

2 CONSUMER_FLUSH_SIZE = 500      # flush theo so luong ban ghi
3 CONSUMER_FLUSH_INTERVAL = 60   # flush theo thoi gian (giay)
4
5 def main_loop(consumer, s3):
6     buffer = defaultdict(list)
7     last_flush = time.time()
8
9     while RUNNING:
10        msg = consumer.poll(timeout=1.0)
11        if msg and not msg.error():
12            row = flatten(json.loads(msg.value()))
13            key = partition_key(row) # (platform, year, month, day)
14            buffer[key].append(row)
15
16        # Kiem tra dieu kien flush kep
17        total = sum(len(v) for v in buffer.values())
18        elapsed = time.time() - last_flush
19        if total >= CONSUMER_FLUSH_SIZE or elapsed >=
20            CONSUMER_FLUSH_INTERVAL:
21            for key, rows in buffer.items():
22                _flush_partition(s3, key, rows)
23            buffer.clear()
24            last_flush = time.time()

```

Listing 8: Logic Dual-trigger Flush của Object Store Writer

Schema Parquet được định nghĩa chặt chẽ bằng Apache Arrow:

```

1 SCHEMA = pa.schema([
2     pa.field("post_id",      pa.string()),
3     pa.field("platform",    pa.string()),
4     pa.field("created_at",   pa.timestamp("ms", tz="UTC")),
5     pa.field("ingested_at",  pa.timestamp("ms", tz="UTC")),
6     pa.field("content",     pa.string()),
7     pa.field("hashtags",    pa.list_(pa.string())),
8     pa.field("likes",       pa.int64()),
9     pa.field("comments",    pa.int64()),
10    pa.field("shares",       pa.int64()),
11    pa.field("views",        pa.int64()),
12    pa.field("raw_json",     pa.string()), # Backup toàn bộ bản ghi
13    ])

```

Listing 9: Schema Parquet chuẩn hóa dữ liệu

Dữ liệu được phân vùng theo platform/year/month/day trên MinIO để hỗ trợ Partition Pruning khi Spark đọc lại.

3.4.1.b Tiền xử lý và Làm sạch dữ liệu Dữ liệu thô từ MinIO thường chứa các vấn đề: bản ghi trùng lặp (do cơ chế At-least-once của Kafka), nội dung thiếu trường bắt buộc, và dữ liệu nằm ngoài khoảng thời gian phân tích. Spark Batch Job thực hiện làm sạch qua 3 bước:

- **Null Handling:** Loại bỏ các bản ghi thiếu trường quan trọng (post_id, platform, created_at, content).

- **Date Filtering:** Lọc dữ liệu trong khoảng thời gian phân tích hợp lệ (tháng 1-4/2026).
- **Deduplication:** Khử trùng lặp dữ liệu dựa trên khóa chính tổ hợp (post_id, platform), giữ lại bản ghi mới nhất theo ingested_at.

```

1 # batch/spark_batch_job.py
2 def clean_data(df: DataFrame) -> DataFrame:
3     # Lọc bản ghi có đủ trường bắt buộc
4     df = df.filter(
5         F.col("post_id").isNotNull() &
6         F.col("platform").isNotNull() &
7         F.col("content").isNotNull() &
8         (F.length(F.col("content")) > 0)
9     )
10    # Lọc theo khoảng thời gian phân tích hợp lệ
11    df = df.filter(
12        (F.col("event_date") >= "2026-01-01") &
13        (F.col("event_date") <= "2026-04-30")
14    )
15    # Khu trùng lặp: giữ bản ghi ingested muộn nhất
16    window = Window.partitionBy("post_id", "platform") \
17        .orderBy(F.col("ingested_at").desc())
18    df = df.withColumn("_rank", F.row_number().over(window)) \
19        .filter(F.col("_rank") == 1) \
20        .drop("_rank")
21    return df

```

Listing 10: Hàm làm sạch dữ liệu Spark

3.4.1.c Xử lý lô và Tổng hợp thống kê (Batch Layer Logic) Tại Cold Path, hệ thống sử dụng PySpark để tổng hợp dữ liệu lịch sử nhằm tìm ra các xu hướng và mẫu từ toàn bộ dataset [3]. Dữ liệu sau khi làm sạch được gom nhóm theo nhiều chiều để tạo ra 5 Batch Views chính:

```

1 platform_daily_stats = cleaned_df.groupBy("platform", "event_date")
2     \
3     .agg(
4         F.count("*").alias("post_count"),
5         F.avg("sentiment_score").alias("avg_sentiment"),
6         F.sum(F.col("likes") + F.col("comments") + F.col("shares"))
7         .alias("total_engagement"),
8         F.countDistinct("author_id").alias("unique_authors")
9     ).orderBy("platform", "event_date")

```

Listing 11: Batch View 1 — Platform Daily Stats

```

1 top_hashtags = cleaned_df \
2     .withColumn("hashtag", F.explode("hashtags")) \
3     .groupBy("platform", "event_week", "hashtag") \
4     .agg(F.count("*").alias("frequency")) \
5     .withColumn("rank", F.row_number().over(
6         Window.partitionBy("platform", "event_week")
7         .orderBy(F.col("frequency").desc())
8     )) \
9     .filter(F.col("rank") <= 100)

```

Listing 12: Batch View 2 — Top Hashtags Weekly


```

1 sentiment_ts = cleaned_df \
2   .withColumn("sentiment_label", F.when(
3     F.col("sentiment_score") > 0.1, "positive"
4   ).when(
5     F.col("sentiment_score") < -0.1, "negative"
6   ).otherwise("neutral")) \
7   .groupBy("platform", "event_hour") \
8   .agg(
9     F.avg("sentiment_score").alias("avg_sentiment"),
10    F.count("*").alias("post_count"),
11    F.sum(F.when(F.col("sentiment_label") == "positive", 1).
12      otherwise(0))
13      .alias("positive_count"),
14    F.sum(F.when(F.col("sentiment_label") == "negative", 1).
15      otherwise(0))
16      .alias("negative_count"),
17  )

```

Listing 13: Batch View 3 — Sentiment Time Series theo giờ

Sentiment Analysis trong Batch Layer: Hệ thống sử dụng hybrid approach — ưu tiên mô hình PhoBERT fine-tuned (nếu artifact tồn tại), ngược lại fallback về Lexicon-based analysis với từ điển song ngữ Anh-Việt và emoji detection:

```

1 # batch/spark_batch_job.py
2 POSITIVE_EMOJI = {"\\U0001F600", "\\U0001F604", "\\U0001F60A", "\\U0001F60D", "\\U0001F970", "\\u2764\\ufe0f", "\\U0001F44D", "\\U0001F525", "\\u2728"}
3 NEGATIVE_EMOJI = {"\\U0001F622", "\\U0001F62D", "\\U0001F621", "\\U0001F620", "\\U0001F494", "\\U0001F44E", "\\U0001F61E"}
4
5 def _lexicon_sentiment_batch(text: str) -> float:
6     if not text:
7         return 0.0
8     normalized = _normalize_text(text)
9     tokens = set(re.findall(r"[a-z0-9_]+", normalized))
10    positive = len(tokens & NORMALIZED_POSITIVE_WORDS)
11    negative = len(tokens & NORMALIZED_NEGATIVE_WORDS)
12    for emoji in POSITIVE_EMOJI:
13        if emoji in text: positive += 1
14    for emoji in NEGATIVE_EMOJI:
15        if emoji in text: negative += 1
16    exclamations = text.count("!")
17    total = positive + negative + 1
18    score = (positive - negative + 0.1 * exclamations) / total
19    return max(-1.0, min(1.0, score))

```

Listing 14: Lexicon-based Sentiment Analysis (fallback)

3.4.2 Xử lý Luồng Thời gian Thực

3.4.2.a Thu thập và Phân phối Sự kiện Thời gian Thực Đây là bước khởi đầu của Hot Path. Speed Layer xử lý từng sự kiện bài đăng ngay khi chúng được phát sinh từ Simulator và đẩy vào Kafka.

```

1 # speed/streaming_job.py

```

```

2 def create_spark() -> SparkSession:
3     builder = (
4         SparkSession.builder
5         .appName("SocialLambdaSpeedLayer")
6         .master(SPARK_MASTER)
7         .config("spark.executor.cores", "1")
8         .config("spark.cores.max", "1") # Giới hạn tài nguyên trên
           Minikube
9     )
10    return configure_s3a(builder).getOrCreate()
11
12 # Đọc streaming từ tất cả 3 Kafka source topics
13 kafka_df = spark.readStream \
14     .format("kafka") \
15     .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP_SERVERS) \
16     .option("subscribe", ",".join(KAFKA_ALL_SOURCE_TOPICS)) \
17     .option("startingOffsets", STREAM_STARTING_OFFSETS) \
18     .option("failOnDataLoss", "false") \
19     .load()

```

Listing 15: Consumer đọc dữ liệu real-time từ Kafka

3.4.2.b NLP Enrichment Pipeline Sau khi nhận được micro-batch từ Spark Streaming (mỗi 5 giây), hàm `foreach_batch` được gọi để làm giàu từng bài đăng với NLP Pipeline:

```

1 # speed/streaming_job.py
2 def foreach_batch(df, batch_id: int) -> None:
3     global _writer, _producer
4     if _writer is None:
5         _writer = RealtimeViewWriter()
6     if _producer is None:
7         _producer = Producer({"bootstrap.servers":
           KAFKA_BOOTSTRAP_SERVERS})
8
9     enriched_posts = []
10    for spark_row in df.collect():
11        row = spark_row.asDict(recursive=True)
12        enrichment = enrich_post(row)
13        enriched = {**row, "enrichment": enrichment}
14        enriched_posts.append(enriched)
15
16    # Publish bài đăng đã enrich lên Kafka topic riêng
17    _producer.produce(
18        "social.enriched.posts",
19        key=str(enriched.get("post_id", "")).encode("utf-8"),
20        value=json.dumps(enriched, ensure_ascii=False,
21                        default=str).encode("utf-8"),
22    )
23
24    _writer.write(enriched_posts)
25    _producer.flush()

```

Listing 16: Hàm `foreach_batch` xử lý enrichment

Cơ chế Fallback của Sentiment Model:

```

1 # speed/nlp_pipeline.py
2 _nlp = None          # spaCy model
3 _sentiment = None    # PhoBERT hoặc fallback
4
5 if pipeline:
6     try:
7         model_path = os.path.join(SENTIMENT_ARTIFACTS_DIR,
8                                   "fine_tuned_phobert")
9         if os.path.exists(os.path.join(model_path, "config.json")):
10             _sentiment = pipeline("sentiment-analysis",
11                                   model=model_path,
12                                   tokenizer=model_path)
13             MODEL_VERSION = "local-fine-tuned-phobert"
14         else:
15             _sentiment = None
16             MODEL_VERSION = "lexicon-fallback"
17     except Exception as exc:
18         logger.warning("Transformer unavailable: %s. Using lexicon.", exc)
19         _sentiment = None

```

Listing 17: Module-level Singleton Loading với multi-layer fallback

3.4.2.c Aggregation Streams với Windowing Kết quả enrichment được ghi vào Redis dưới dạng cửa sổ thời gian (time windows):

```

1 # speed/realtime_stores.py
2 class RealtimeViewWriter:
3     def _write_to_redis(self, posts: list[dict]) -> None:
4         if not self.redis:
5             return
6         pipe = self.redis.pipeline()
7         for post in posts:
8             platform = post.get("platform", "unknown")
9             hour_key = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H")
10
11             # Cập nhật stats theo cửa sổ 1 giờ
12             stats_key = f"rt:stats:{platform}:{hour_key}"
13             pipe.hincrby(stats_key, "post_count", 1)
14             pipe.expire(stats_key, 7200) # TTL: 2 giờ
15
16             # Cập nhật top hashtags (Sorted Set)
17             hashtags_key = f"rt:hashtags:{platform}:{hour_key}"
18             for tag in (post.get("hashtags") or []):
19                 pipe.zincrby(hashtags_key, 1, tag.lower())
20             pipe.expire(hashtags_key, 3600) # TTL: 1 giờ
21
22         pipe.execute()

```

Listing 18: Cập nhật stats theo cửa sổ thời gian 1 giờ vào Redis

3.4.3 Cơ chế Tự động Huấn luyện lại mô hình (ML Retraining Pipeline)

Bên cạnh luồng xử lý lô (Batch ETL), hệ thống duy trì một luồng MLOps hoàn chỉnh thông qua DAG `virality_retrain_dag.py`. Định kỳ, luồng này sẽ lấy dữ liệu mới

nhất (với các feature engineering từ lịch sử tương tác của người dùng) để huấn luyện lại mô hình Machine Learning dự đoán độ lan truyền (Virality Prediction).

Đặc biệt, hệ thống triển khai cơ chế **Safe Artifacts** (`ml/virality/safe_artifacts.py`). Khi quá trình huấn luyện hoàn tất, mô hình mới chỉ được tự động triển khai (promote) lên môi trường Production nếu các chỉ số đánh giá (metrics như RMSE hay Accuracy) vượt qua ngưỡng an toàn hoặc tốt hơn mô hình hiện tại, đảm bảo hệ thống không bị ảnh hưởng tiêu cực (model degradation).

3.4.4 Lưu trữ vào Elasticsearch (Realtime Sink)

Song song với Redis, mỗi bài đăng được index vào Elasticsearch để hỗ trợ full-text search và date-range filter trên Dashboard Real-time View:

```
1 # speed/realtime_stores.py
2 def _write_elasticsearch(self, posts: list[dict]) -> None:
3     docs = []
4     for post in posts:
5         enrichment = post.get("enrichment") or {}
6         metrics = post.get("metrics") or post.get("engagement")
7             or {}
8         docs.append({
9             "doc_id": f"realtime:{post.get('post_id')}",
10            "post_id": post.get("post_id"),
11            "platform": post.get("platform"),
12            "content": post.get("content"),
13            "hashtags": post.get("hashtags") or [],
14            "event_ts": _created_at_to_dt(post.get("created_at")).isoformat(),
15            "engagement_score": (int(metrics.get("likes") or 0)
16                                + int(metrics.get("comments") or 0) * 2
17                                + int(metrics.get("shares") or 0) * 3),
18            "sentiment_score": enrichment.get("sentiment_score"),
19            "sentiment_label": enrichment.get("sentiment_label"),
20            "keywords": enrichment.get("keywords") or [],
21            "entities": enrichment.get("entities") or [],
22            "language": enrichment.get("language"),
23            "indexed_at": datetime.now(timezone.utc).isoformat(),
24        })
25     self.es.bulk_index(ES_REALTIME_INDEX, docs)
```

Listing 19: Ghi bài đăng đã enriched vào Elasticsearch Realtime Index

Mỗi document được đánh index với `doc_id = "realtime:{post_id}"`, cho phép ES tự động upsert khi cùng một bài đăng được cập nhật bởi một batch mới.

3.4.5 Merge Service — Hợp nhất Batch và Speed View

ServeQuery là thành phần trung tâm của Serving Layer, thực hiện merge kết quả từ hai lớp theo cơ chế **Read-time Merge**:

```
1 # serving/merge_service.py
2 class ServeQuery:
```

```

3     def get_posts(self, platform=None, start=None, end=None,
4         limit=50) -> list[dict]:
5         # Buoc 1: Truy van Elasticsearch Batch Index
6         batch_posts = self._search(
7             index=ES_BATCH_INDEX,
8             query=self._build_query(platform, start, end),
9             size=limit
10        )
11        # Buoc 2: Truy van Elasticsearch Realtime Index
12        rt_posts = self._search(
13            index=ES_REALTIME_INDEX,
14            query=self._build_query(platform, start, end),
15            size=limit
16        )
17        # Buoc 3: Merge va dedup theo post_id
18        merged: dict[str, dict] = {}
19        for post in batch_posts + rt_posts:
20            pid = post.get("post_id")
21            if pid not in merged:
22                merged[pid] = post
23            else:
24                existing_ts = self._parse_event_ts(merged[pid])
25                new_ts = self._parse_event_ts(post)
26                if new_ts > existing_ts:
27                    merged[pid] = post
28
29        return sorted(merged.values(),
30                        key=lambda p: self._parse_event_ts(p),
31                        reverse=True)[:limit]

```

Listing 20: Merge Service hợp nhất Batch và Realtime views

3.5 Kiểm thử và Đảm bảo chất lượng (QA)

Hệ thống được thiết kế với tư duy kiểm thử (Testing) cực kỳ chặt chẽ nhằm đảm bảo độ tin cậy tuyệt đối khi triển khai trên môi trường Production. Thư mục `tests/` bao gồm các cấp độ kiểm thử toàn diện:

- **Unit & Integration Test:** Kiểm thử chi tiết từng hàm, module lõi (như các API Endpoints, Elasticsearch queries, logic của NLP Pipeline và ML Retrain).
- **End-to-End (E2E) Test:** Các kịch bản như `test_pipeline_contract.py` được thực thi để đảm bảo schema không bao giờ bị vỡ hay rò rỉ dữ liệu giữa các thành phần — từ Ingestion Layer cho đến khi lưu trữ xuống Data Lake.
- **Load Test (Kiểm thử chịu tải):** Sử dụng công cụ **Locust** (`locustfile.py`), nhóm giả lập hàng ngàn người dùng truy cập đồng thời vào hệ thống API. Việc này giúp đánh giá sát sao giới hạn phần cứng và hỗ trợ tinh chỉnh tài nguyên cấp phát (Resource Limits/Requests) hợp lý cho các Kubernetes Pods.

3.6 Thiết lập giám sát

Để đảm bảo khả năng quan sát (Observability), hệ thống thiết lập các cổng giám sát thời gian thực thông qua cơ chế Port-forwarding của Kubernetes:

```

1 # 1. Airflow UI - Điều phối DAG Batch Pipeline

```

```

2 kubectl port-forward svc/airflow-webserver 8085:8080 -n social-
  pipeline
3
4 # 2. Kafka UI - Giám sát topics và consumer groups
5 kubectl port-forward svc/kafka-ui 8086:8080 -n social-pipeline
6
7 # 3. MinIO Console - Quản lý Data Lake
8 kubectl port-forward svc/minio 9001:9001 -n social-pipeline
9
10 # 4. Spark UI - Giám sát Spark Jobs
11 kubectl port-forward svc/spark-master 8080:8080 -n social-pipeline
12
13 # 5. Elasticsearch - Kiểm tra cluster health
14 kubectl port-forward svc/elasticsearch 9200:9200 -n social-pipeline

```

Listing 21: Port-forward để truy cập giao diện giám sát

Các giao diện quản trị cung cấp cái nhìn chi tiết về trạng thái hệ thống:

- **Airflow UI (<http://localhost:8085>):** Giám sát trạng thái DAG `social_lambda_batch_pipeline`, xem log thực thi từng Task và kích hoạt chạy lại (Retry) khi có lỗi. Airflow cũng tích hợp Slack Alert callback để thông báo khi pipeline thất bại.

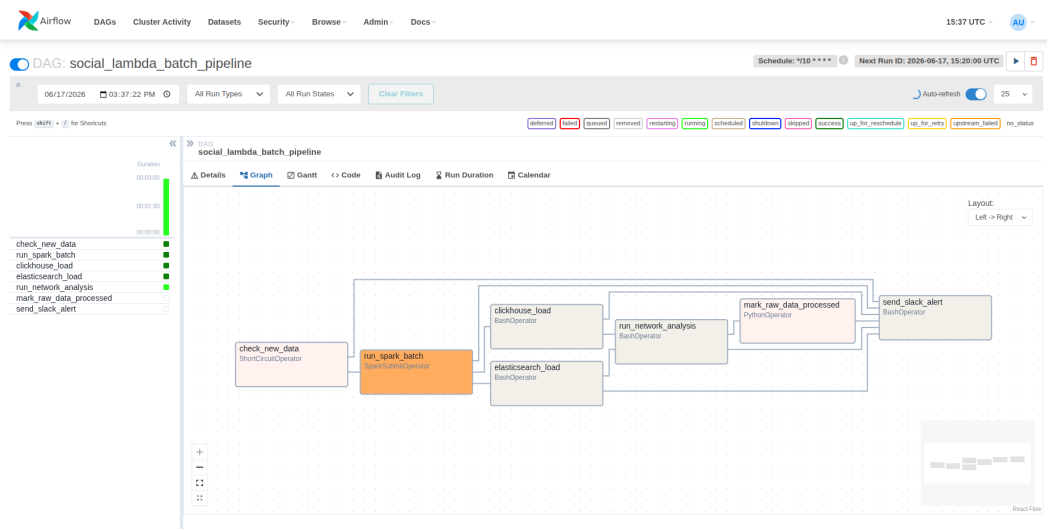


Figure 2: Giao diện Apache Airflow giám sát DAG Batch Pipeline.

- **Kafka UI (<http://localhost:8086>):** Giám sát các Kafka topic chính như `social.reddit.posts`, `social.facebook.posts`, `social.instagram.posts`, `social.enriched.posts` và `social.dlq`; đồng thời theo dõi partition, message offset và consumer groups.

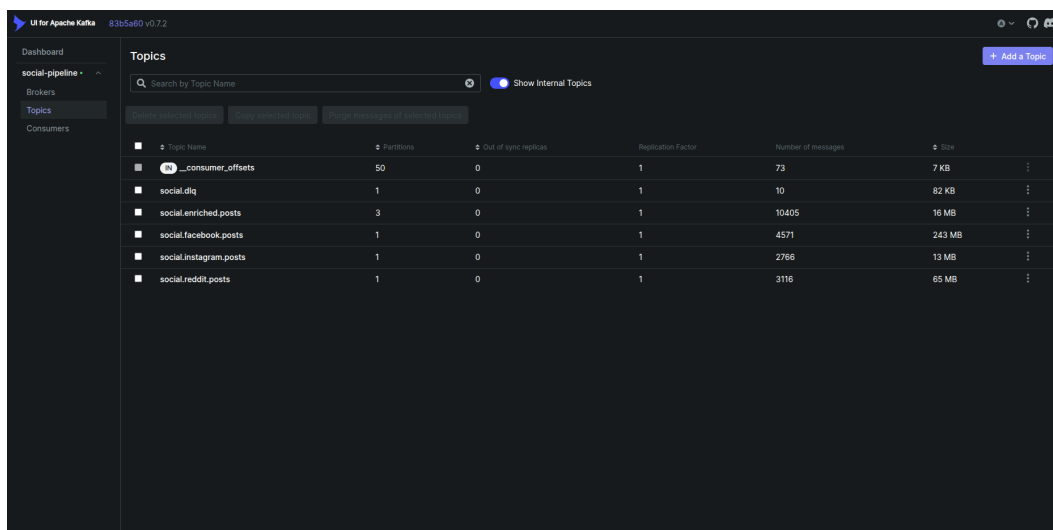


Figure 3: Giao diện Kafka UI theo dõi các topic và trạng thái streaming.

- **MinIO Console (<http://localhost:9001>):** Quản lý Data Lake bucket `social-lake`, kiểm tra cấu trúc phân vùng Parquet (`data/raw/platform/year/month/day/`), và theo dõi dung lượng lưu trữ theo nền tảng.

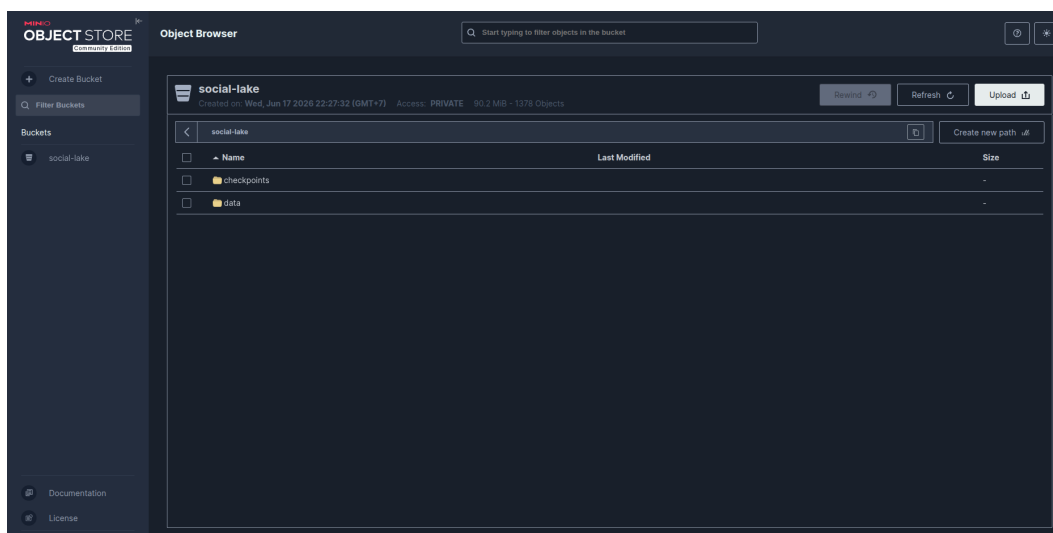


Figure 4: Giao diện MinIO Console quản lý bucket và dữ liệu Parquet trong Data Lake.

- **Spark UI (<http://localhost:8080>):** Theo dõi Spark Jobs đang chạy, DAG của execution plan, số lượng Stage và Task, thông số bộ nhớ executor (Heap used, GC time), và phát hiện Data Skew.
- **Elasticsearch Cluster Health:** Kiểm tra trạng thái index, shard allocation, và document count qua REST API:

```

1 # Kiểm tra sức khỏe cluster
2 curl http://localhost:9200/_cluster/health?pretty
3
4 # Thống kê index realtime
5 curl http://localhost:9200/social_realtime_views/_stats?pretty

```

Listing 22: Kiểm tra health của Elasticsearch cluster

- **Prometheus Metrics (<http://localhost:8000/metrics>):** FastAPI server expose các metrics Prometheus: số lượng bản ghi đã publish (`records_published_total`), lỗi publish (`publish_errors_total`), và latency phân phối (`publish_latency_seconds`). Không có observability = không thể debug production issues.

3.7 Giao diện người dùng và Trực quan hóa kết quả

Hệ thống cung cấp giao diện Dashboard tương tác giúp người vận hành theo dõi toàn diện dữ liệu mạng xã hội. Giao diện được chia thành hai phân hệ chính theo nguồn dữ liệu.

3.7.1 Phân hệ Giám sát Thời gian thực (Real-time View)

Module này kết nối trực tiếp với Elasticsearch Realtime Index và Redis để hiển thị dữ liệu với độ trễ thấp (cập nhật mỗi 5-10 giây). Giao diện bao gồm:

1. **Bảng bài đăng thời gian thực:** Hiển thị danh sách bài đăng mới nhất từ cả 3 nền tảng, kèm nhãn cảm xúc (Positive/Negative/Neutral) được gán bởi NLP Pipeline. Hỗ trợ lọc theo nền tảng và khoảng thời gian.

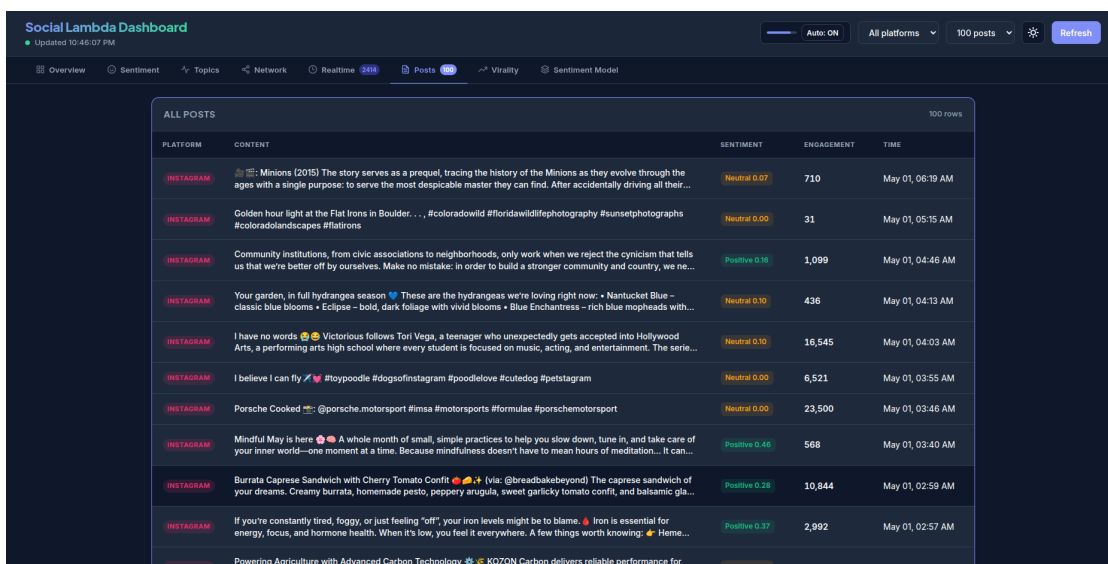


Figure 5: Dashboard Real-time View hiển thị bảng bài đăng thời gian thực, KPI và hashtag nổi bật.

2. **Thẻ thống kê realtime:** Các KPI tổng hợp theo cửa sổ thời gian (1 giờ, 24 giờ): tổng số bài đăng, số bài đăng trên mỗi nền tảng, điểm cảm xúc trung bình.
3. **Biểu đồ Top Hashtags theo thời gian thực:** Sorted Set từ Redis được trực quan hóa dạng bar chart, tự động cập nhật khi có hashtag mới nổi bật.

Ví dụ API endpoint phục vụ Real-time View:

```
1 GET /api/v1/stats/realtime?platform=facebook&window_hours=1
2 GET /api/v1/hashtags/top?platform=reddit&limit=10
```

Listing 23: API endpoints phục vụ Real-time Dashboard

3.7.2 Phân hệ Phân tích Lịch sử (Historical View)

Dữ liệu tổng hợp từ Batch Layer được tổ chức thành 4 tab chuyên biệt, cung cấp các góc nhìn từ tổng quan đến chi tiết:

- **Tab Tổng quan (Overview):** Biểu đồ xu hướng cảm xúc theo ngày trên từng nền tảng (Line Chart), tổng số bài đăng theo nền tảng (Bar Chart), và phân phối nhân cảm xúc (Donut Chart).

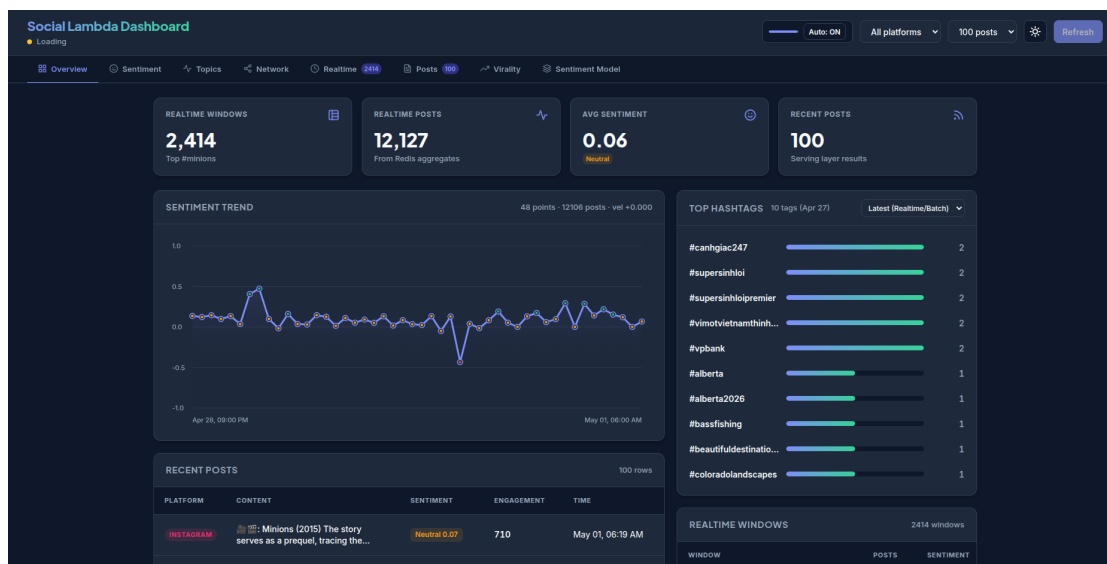


Figure 6: Tab Tổng quan hiển thị xu hướng cảm xúc, số lượng bài đăng và phân phối sentiment theo nền tảng.

- **Tab Hashtag Analysis:** Top 20 hashtag trên mỗi nền tảng dưới dạng Word Cloud hoặc Ranked List, hỗ trợ so sánh cross-platform.
- **Tab Network Analysis:** Trực quan hóa mạng lưới tương tác tác giả dưới dạng Force-directed Graph. Kích thước node thể hiện PageRank Score (ảnh hưởng), màu sắc thể hiện cộng đồng (Louvain community). Hỗ trợ truy vấn Top-N influencers theo nền tảng.
- **Tab ML Insights:** Quản lý và kiểm tra trạng thái mô hình Machine Learning. Cho phép kích hoạt quá trình huấn luyện lại mô hình Virality Prediction và Sentiment từ giao diện, theo dõi log training theo thời gian thực.

API endpoints phục vụ Historical View:

```
1 GET /api/v1/sentiment/trend?platform=instagram&granularity=day
2 GET /api/v1/network/graph?platform=reddit&limit=200
3 GET /api/v1/network/pagerank?platform=facebook&top_n=10
4 GET /api/v1/topics/distribution
```

Listing 24: API endpoints phục vụ Historical Dashboard

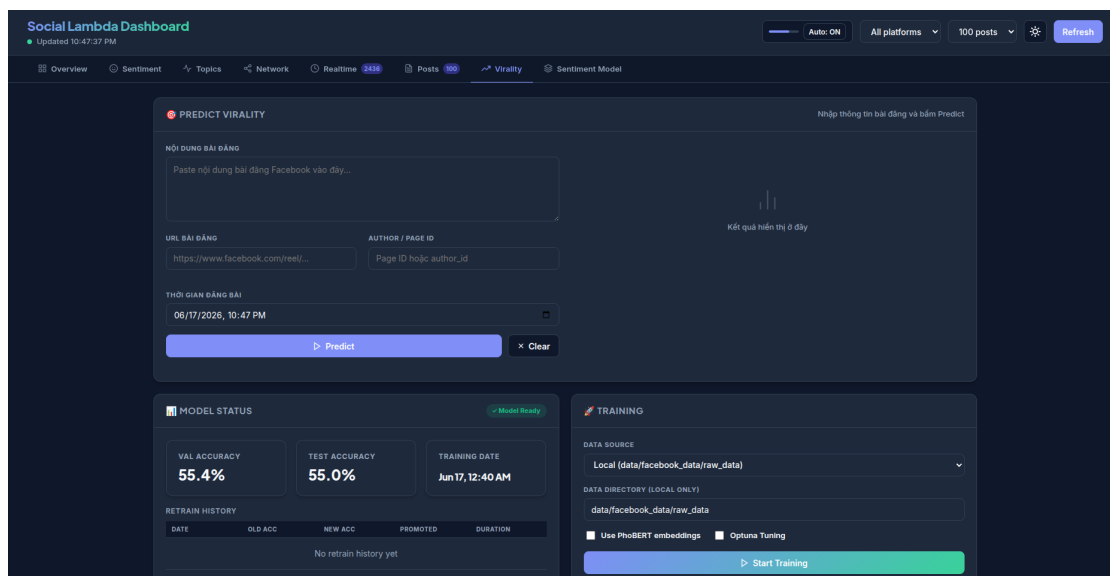


Figure 7: Tab ML Insights theo dõi quá trình huấn luyện và đánh giá mô hình Virality Prediction.

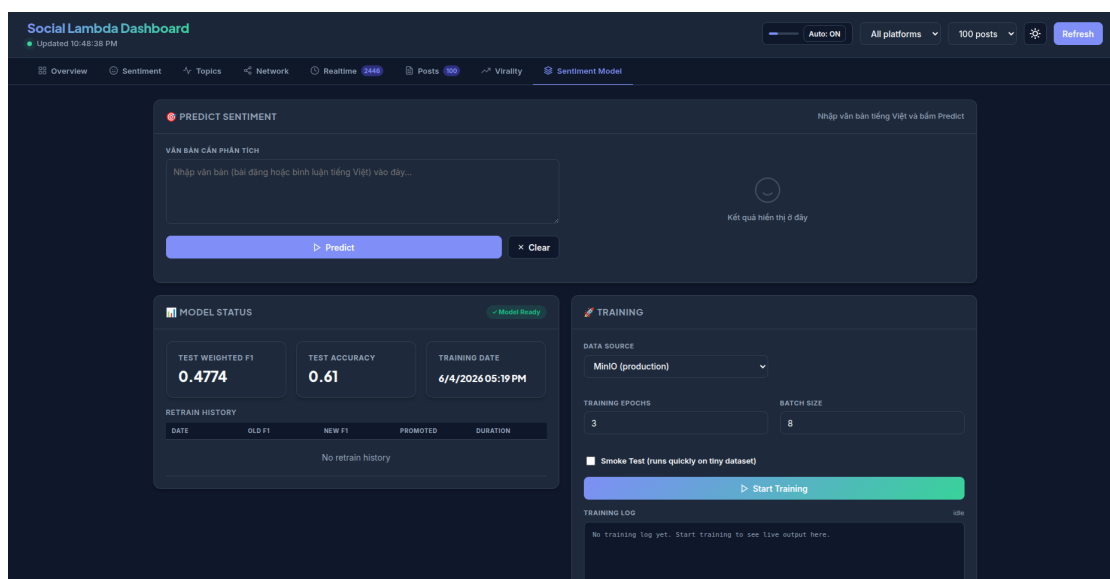


Figure 8: Tab ML Insights quản lý mô hình Sentiment Analysis và kết quả đánh giá cảm xúc.

4 Bài học kinh nghiệm

Trong quá trình thiết kế và triển khai hệ thống Big Data phân tích mạng xã hội, nhóm đã gặp phải nhiều thách thức kỹ thuật. Dưới đây là các bài học quan trọng nhất được rút ra, tuân theo quy trình giải quyết vấn đề thực tế.

4.1 Bài học 1: Spark Out-Of-Memory khi tích hợp NLP vào Batch Job

Mô tả vấn đề

- **Bối cảnh:** Spark Batch Job (`spark_batch_job.py`) thực hiện phân tích cảm xúc trên toàn bộ bài đăng bằng cách đăng ký UDF gọi vào thư viện Transformer (PhoBERT). Job đọc toàn bộ dữ liệu Parquet từ MinIO (3 nền tảng $\times$ 4 tháng) vào một DataFrame duy nhất trước khi xử lý.
- **Thách thức:**
 1. **Driver OOM:** Spark mặc định sử dụng `spark.sql.shuffle.partitions=200`, tạo ra hàng trăm partition nhỏ. Khi DataFrame lớn được shuffle, Driver node phải điều phối metadata của hàng trăm task đồng thời, dẫn đến `java.lang.OutOfMemoryError: Java heap space`.
 2. **Executor OOM:** Hàm UDF gọi PhoBERT model load toàn bộ model weights ($\sim 500\text{MB}$) vào bộ nhớ executor. Với nhiều Executor Worker chạy đồng thời, tổng bộ nhớ vượt quá giới hạn Pod (1GB/executor pod) gây OOMKilled.
 3. **Recomputation overhead:** DataFrame chưa được cache nên mỗi lần tính toán Batch View đều phải đọc lại toàn bộ dữ liệu từ MinIO.
- **Tác động hệ thống:** Spark Job liên tục thất bại sau 10-15 phút, Dashboard Historical tab không có dữ liệu cập nhật.

Các phương pháp tiếp cận

- **Phương pháp 1: Vertical Scaling.** Tăng memory limit của Spark Worker Pod từ 1 GB lên 4 GB trong Kubernetes manifest.
Hạn chế: Không giải quyết nguyên nhân gốc rễ. Trên môi trường Minikube tổng RAM hạn chế (10 GB), việc cấp quá nhiều cho Spark Worker sẽ thiếu tài nguyên cho Kafka, Elasticsearch, MinIO.
- **Phương pháp 2: Reduce Shuffle Partitions tĩnh.** Cấu hình `spark.sql.shuffle.partitions`
Hạn chế: Cấu hình tĩnh không phản ánh được kích thước dữ liệu thực tế. Nếu dữ liệu tăng lên, cần chỉnh lại thủ công.

Giải pháp thực tế

Nhóm kết hợp 4 biện pháp tối ưu đồng thời:

```
1 # batch/spark_batch_job.py
2 DEFAULT_BATCH_INPUT_PARTITIONS = int(os.getenv("
   BATCH_INPUT_PARTITIONS", "8"))
3 DEFAULT_BATCH_SHUFFLE_PARTITIONS = int(os.getenv("
   BATCH_SHUFFLE_PARTITIONS", "4"))
4
5 spark.conf.set("spark.sql.shuffle.partitions",
6               str(DEFAULT_BATCH_SHUFFLE_PARTITIONS))
7 df = spark.read.parquet(raw_path).repartition(
   DEFAULT_BATCH_INPUT_PARTITIONS)
```

Listing 25: Giảm số lượng partition cho Minikube

```
1 # Persist xuống disk để giảm áp lực heap memory
2 cleaned_df = cleaned_df.persist(StorageLevel.DISK_ONLY)
3
4 # Tất cả 5 Batch Views đều đọc từ cached DataFrame
5 platform_stats = _compute_platform_daily_stats(cleaned_df)
6 top_hashtags = _compute_top_hashtags(cleaned_df)
7 sentiment_ts = _compute_sentiment_time_series(cleaned_df)
```

```

8 author_activity = _compute_author_activity(cleaned_df)
9 top_posts      = _compute_top_posts(cleaned_df)
10
11 cleaned_df.unpersist()

```

Listing 26: Persist DataFrame để tránh recomputation

```

1 spark = SparkSession.builder \
2     .config("spark.memory.fraction", "0.7") \
3     .config("spark.memory.storageFraction", "0.3") \
4     .config("spark.executor.memory", "2g") \
5     .config("spark.driver.memory", "1g") \
6     .getOrCreate()

```

Listing 27: Cấu hình Spark Memory Fraction

Kết quả: Spark Batch Job chạy ổn định với thời gian thực thi ~20 phút trên 1 CPU core. Tổng I/O giảm đáng kể do persist tránh recomputation (đọc MinIO 1 lần thay vì 5 lần).

Bài học kinh nghiệm (Key Takeaways)

- **Right-size Partitions:** Số shuffle partitions mặc định (200) được thiết kế cho cluster lớn. Trên môi trường Minikube, cần giảm xuống phù hợp với số CPU core thực tế — nguyên tắc: $\text{shuffle partitions} \approx 2-4 \times \text{số executor cores}$.
- **Persist is not free:** Việc persist DataFrame tiêu tốn disk I/O, nhưng lợi ích tránh recomputation thường vượt trội nếu DataFrame được tái sử dụng nhiều lần. Luôn `unpersist()` sau khi dùng xong.
- **Tách biệt ML inference khỏi Batch ETL:** Batch Layer nên ưu tiên throughput và idempotency. Inference nặng (PhoBERT) nên được thực hiện ở Speed Layer.

4.2 Bài học 2: Kubernetes Pod Crash do Resource Limits không phù hợp

Mô tả vấn đề

- **Bối cảnh:** Hệ thống triển khai nhiều thành phần có nhu cầu bộ nhớ cao đồng thời trên Minikube 10GB: Kafka (JVM), Spark Master/Worker (JVM + Python), Elasticsearch (JVM + Lucene), Airflow Scheduler, NLP containers (PyTorch).
- **Thách thức:**
 1. **Pod OOMKilled:** Kubernetes Kubelet force kill container khi vượt quá `limits.memory` với exit code 137 (SIGKILL).
 2. **CrashLoopBackOff:** Kubernetes áp dụng exponential backoff delay (10s → 20s → 40s → 80s → 300s) gây pod ở trạng thái không phục vụ được.
 3. **Resource Starvation:** Cascade failure — Elasticsearch OOM → dữ liệu realtime không được index → Dashboard trống.
 4. **Thiếu Readiness/Liveness Probe:** Kubernetes gửi traffic vào pod chưa sẵn sàng, gây lỗi connection refused.
- **Tác động hệ thống:** Pipeline dừng hoàn toàn khi Spark Worker hoặc Airflow bị crash. Debug khó khăn vì pod bị kill đột ngột, không để lại log đầy đủ.

Các phương pháp tiếp cận

- **Phương pháp 1: Không đặt Resource Limits.** Chỉ có requests, không có limits.
Hạn chế: Một pod có thể “ăn” toàn bộ RAM của node, gây OOM ở mức OS.
- **Phương pháp 2: Đặt limits rất thấp.** Đặt limits bằng với requests.
Hạn chế: JVM-based services (Kafka, ES, Spark) thường cần burst memory cao hơn steady-state — đặt thấp sẽ gây OOMKilled ngay khi load cao.

Giải pháp thực tế

Nhóm áp dụng chiến lược **Tiered Resource Management**:

```
1 # k8s/03-infrastructure/kafka.yaml
2 resources:
3   requests:
4     cpu: 500m
5     memory: 1Gi
6   limits:
7     cpu: 1000m
8     memory: 2Gi    # Cho phép burst gap đôi khi xử lý throughput cao
```

Listing 28: Resource Limits cho Kafka (Tier 1 — Infrastructure)

```
1 # k8s/04-apps/spark-worker.yaml
2 resources:
3   requests:
4     cpu: 500m
5     memory: 1Gi
6   limits:
7     cpu: 2000m
8     memory: 3Gi    # Dư cho shuffle + PhoBERT model weights (~500MB)
```

Listing 29: Resource Limits cho Spark Worker (Tier 3 — Heavy)

```
1 readinessProbe:
2   httpGet:
3     path: /
4     port: 8080
5   initialDelaySeconds: 10
6   periodSeconds: 10
7   timeoutSeconds: 5
8   failureThreshold: 5
9
10 livenessProbe:
11   httpGet:
12     path: /
13     port: 8080
14   initialDelaySeconds: 30
15   periodSeconds: 30
16   failureThreshold: 3
```

Listing 30: Readiness và Liveness Probe cho Spark Master

```
1 apiVersion: v1
2 kind: ResourceQuota
3 metadata:
4   name: social-pipeline-quota
```

```

5 namespace: social-pipeline
6 spec:
7   hard:
8     requests.memory: 8Gi
9     limits.memory: 12Gi
10    requests.cpu: "4"
11    limits.cpu: "8"

```

Listing 31: Namespace ResourceQuota kiểm soát tổng tài nguyên

Kết quả

- Pod OOMKilled giảm từ xảy ra hàng ngày xuống còn rất hiếm khi Spark Job chạy với dữ liệu thông thường.
- CrashLoopBackOff không còn xảy ra với infrastructure services trong điều kiện vận hành bình thường.
- Readiness Probe giúp Kubernetes tránh gửi traffic vào pod chưa sẵn sàng, giảm lỗi connection refused ở API layer.
- Thời gian khởi động toàn bộ hệ thống giảm từ ~15 phút xuống ~8 phút nhờ scheduling hiệu quả hơn.

Bài học kinh nghiệm (Key Takeaways)

- **Requests là cam kết, Limits là trần:** Kubernetes schedule pod dựa trên **requests**, không phải **limits**. Đặt **requests** quá thấp khiến Kubernetes không dự báo được resource contention.
- **JVM Services cần burst headroom:** Kafka, Spark, Elasticsearch dùng JVM với GC có tính burst. Limits nên đặt gấp 1.5-2× so với steady-state memory usage.
- **Probe Design là bắt buộc, không phải tùy chọn:** Mọi container đều cần ít nhất **readinessProbe**. Không có probe, Kubernetes không thể phân biệt pod “đang khởi động” với pod “đã crash”.
- **Debug OOMKilled:** Kiểm tra `kubectl describe pod <name>` để xem **OOMKilled: true** và `kubectl get events` để tìm chuỗi sự kiện dẫn đến crash.

4.3 Bài học 3: Architectural Pivot — Revert Merge Service từ ClickHouse về Elasticsearch/Redis

Mô tả vấn đề

- **Bối cảnh:** Trong thiết kế ban đầu, nhóm triển khai ClickHouse làm backend chính cho cả Batch Views và Serving Layer. Merge Service (**ServeQuery**) query trực tiếp ClickHouse để lấy batch views, kết hợp với Elasticsearch realtime index để merge.
- **Thách thức:**
 1. **Query Latency không đồng đều:** ClickHouse được tối ưu cho analytics queries (scan hàng triệu row, aggregate, GROUP BY). Merge Service cần các queries nhỏ, point-lookup theo `post_id` với latency < 100ms. ClickHouse có cold-start overhead khiến các queries đầu tiên chậm hơn nhiều.

2. **Phức tạp trong Merge Logic:** Merge giữa ClickHouse và Elasticsearch đòi hỏi hai client library, hai kiểu query language khác nhau.
 3. **Operational Overhead:** Duy trì hai hệ thống lưu trữ cho cùng mục đích tăng số điểm thất bại.
 4. **Schema Evolution:** ClickHouse sử dụng `ReplacingMergeTree` với partition tháng. Khi cần thêm cột mới vào batch views, phải thực hiện `ALTER TABLE`.
- **Tác động hệ thống:** API endpoint `/api/v1/posts` có latency 500ms-2 giây, chậm hơn 10× so với mục tiêu 100-200ms, Dashboard bị lag khi load trang.

Các phương pháp tiếp cận

- **Phương pháp 1: Giữ ClickHouse, tối ưu queries.** Thêm caching layer (Redis) trước ClickHouse.
Hạn chế: Thêm một lớp phức tạp nữa (cache invalidation logic). Vẫn cần duy trì ClickHouse cho analytics.
- **Phương pháp 2: Revert về đồng nhất Elasticsearch.** Batch Views cũng được index vào Elasticsearch (index `social_batch_views`), Merge Service chỉ cần một client duy nhất.

Giải pháp thực tế

Nhóm quyết định `revert merge_service.py` về sử dụng Elasticsearch và Redis làm backend duy nhất cho Serving Layer. ClickHouse vẫn được giữ lại nhưng chỉ phục vụ mục đích Data Warehouse OLAP analytics, không tham gia vào critical serving path:

```

1 # serving/merge_service.py
2 class ServeQuery:
3     def __init__(self, es_host=ES_HOST, redis_host=REDIS_HOST, ...):
4         :
5         self.es_host = es_host
6         self._redis = redis.Redis(host=redis_host, port=redis_port)
7
8     def get_posts(self, ...):
9         # Batch posts: tu ES index "social_batch_views"
10        batch_posts = self._search(index=ES_BATCH_INDEX, ...)
11        # Realtime posts: tu ES index "social_realtime_views"
12        rt_posts = self._search(index=ES_REALTIME_INDEX, ...)
13        # Merge + dedup: logic đồng nhất, cùng schema
14        return self._merge_and_dedup(batch_posts + rt_posts)
15
16    def get_realtime_stats(self, platform, window_hours=1):
17        # Stats thực su realtime: tu Redis (sub-millisecond)
18        hour_key = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H")
19        stats_key = f"rt:stats:{platform}:{hour_key}"
20        return self._redis.hgetall(stats_key)

```

Listing 32: Merge Service sau khi revert về Elasticsearch + Redis

Phân chia rõ trách nhiệm sau khi revert:

Thành phần	Vai trò
Redis	Stats tổng hợp realtime — latency <1 ms
Elasticsearch Realtime	Full-text search bài đăng mới nhất — latency 20–50 ms
Elasticsearch Batch	Historical views đã tổng hợp — latency 50–150 ms
ClickHouse	Analytics nặng, không trong critical path của API

Kết quả: API latency giảm từ 500ms-2s xuống còn 50-150ms. Merge logic đơn giản hơn đáng kể (1 client library thay vì 2). Số lượng dịch vụ trong critical serving path giảm từ 3 (ClickHouse + ES + Redis) xuống còn 2 (ES + Redis).

Bài học kinh nghiệm (Key Takeaways)

- **Choose tools by access pattern, not familiarity:** ClickHouse xuất sắc cho OLAP analytics, không phải cho low-latency point queries.
- **Homogeneous serving layer giảm complexity:** Khi batch views và realtime views đều dùng cùng một backend, merge logic trở nên đồng nhất và dễ test hơn.
- **Không sợ revert kiến trúc:** Một quyết định kiến trúc sai được phát hiện sớm và sửa chữa luôn tốt hơn tiếp tục build trên một nền tảng không phù hợp.

4.4 Bài học 4: NLP Pipeline Cold-Start và PhoBERT Loading Overhead trong Speed Layer

Mô tả vấn đề

- **Bối cảnh:** Speed Layer sử dụng PhoBERT (~500MB weights) để phân tích cảm xúc bài đăng trong `nlp_pipeline.py`. Model được load khi Python process khởi động.
- **Thách thức:**
 1. **Cold-start latency:** Lần đầu tiên Spark Streaming Worker pod khởi động (hoặc sau khi bị OOMKilled), PhoBERT cần 30-60 giây để load từ disk vào RAM. Trong khoảng thời gian này, NLP enrichment chưa sẵn sàng.
 2. **Model loaded per-process, not per-request:** Nếu không có Singleton pattern, mỗi lần gọi `enrich_post()` có thể cố load lại model.
 3. **Memory không đủ cho cả Spark JVM và PyTorch:** PhoBERT (~500MB) cộng Spark JVM overhead (~1GB) dễ vượt memory limit của pod.
- **Tác động hệ thống:** Bài đăng trong 1–2 micro-batch đầu tiên sau khi pod khởi động không có enrichment data. Dashboard hiển thị các bài đăng không có nhãn cảm xúc.

Các phương pháp tiếp cận

- **Phương pháp 1: Load model trong mỗi batch.** Load PhoBERT mỗi khi `foreach_batch` được gọi.
Hạn chế: 30–60 giây overhead mỗi 5 giây là không chấp nhận được.
- **Phương pháp 2: Pre-warm container bằng init container.**
Hạn chế: Tăng thời gian pod startup. Vẫn không giải quyết khi pod bị restart đột

ngột.

Giải pháp thực tế

Nhóm áp dụng **Module-level Singleton Loading** với **multi-layer fallback** và biến môi trường cho lightweight mode:

```
1 # Cho phép force fallback về lexicon để tiết kiệm RAM trên Minikube
2 is_lightweight = os.getenv("SPEED_LIGHTWEIGHT", "false").lower() \
3     in ("1", "true", "yes")
4 if is_lightweight:
5     _sentiment = None
6     MODEL_VERSION = "lexicon-fallback-forced"
```

Listing 33: Biến môi trường cho chế độ nhẹ tiết kiệm RAM

```
1 def _analyze_sentiment(text: str) -> dict:
2     if _sentiment and text:
3         try:
4             result = _sentiment(text[:512], truncation=True)[0]
5             score = result["score"] if result["label"] == "POSITIVE"
6             \
7             else -result["score"]
8             return {"score": score, "label": result["label"].lower
9             (),
10                 "model": MODEL_VERSION}
11         except Exception as exc:
12             logger.warning("Transformer inference failed: %s", exc)
13             # Fallback về lexicon bất kỳ
14             score = _lexicon_sentiment(text)
15             return {"score": score,
16                 "label": "positive" if score > 0.1 else
17                     "negative" if score < -0.1 else "neutral",
18                 "model": "lexicon-fallback"}
```

Listing 34: Hàm sentiment đảm bảo luôn có output

Kết quả: Mọi bài đăng đều nhận được enrichment output. Trên môi trường Minikube resource-limited, `SPEED_LIGHTWEIGHT=true` cho phép chạy Speed Layer với footprint bộ nhớ ~200MB thay vì ~700MB.

Bài học kinh nghiệm (Key Takeaways)

- **Module-level initialization là Singleton pattern tự nhiên trong Python:** Python import system đảm bảo module chỉ được execute 1 lần per interpreter process.
- **Graceful degradation là yêu cầu thiết kế, không phải afterthought:** Một hệ thống production không bao giờ nên crash vì model không available. Luôn thiết kế fallback logic trước khi viết happy path.
- **Environment flag cho lightweight mode:** Biến môi trường `SPEED_LIGHTWEIGHT` cho phép switch giữa chế độ đầy đủ (PhoBERT) và chế độ nhẹ (lexicon) mà không cần deploy lại code.

4.5 Bài học 5: Small Files Problem — Chiến lược Flush Kafka → MinIO

Mô tả vấn đề

- **Bối cảnh:** object-store-writer là Kafka Consumer liên tục, poll dữ liệu từ Kafka và ghi xuống MinIO. Thiết kế ban đầu: flush mỗi khi nhận được 1 message.
- **Thách thức:**
 1. **Small Files Problem:** Simulator đẩy dữ liệu theo burst (20 records/giây, mỗi record ~2-5KB), mỗi flush tạo ra một file Parquet kích thước vài KB. Sau vài giờ chạy, MinIO chứa hàng chục nghìn file nhỏ.
 2. **Spark Batch Job chậm khi list files:** Spark phải gọi LIST objects API của MinIO. Với hàng chục nghìn file, bước list này mất vài phút.
 3. **Overhead per-write API call:** Ghi 20 file/giây $\times$ 3 nền tảng = 60 API calls/giây gây tải lớn cho MinIO.
- **Tác động hệ thống:** Airflow DAG task run_spark_batch timeout sau 30 phút, MinIO CPU usage cao bất thường.

Các phương pháp tiếp cận

- **Phương pháp 1: Compaction Job hậu kỳ.** Chạy một Spark Job định kỳ để merge file nhỏ lại.
Hạn chế: Tăng độ phức tạp pipeline, tốn kém I/O gấp đôi (đọc file nhỏ + ghi file lớn).
- **Phương pháp 2: Tăng flush threshold.** Chỉ flush khi tích lũy đủ N records hoặc đủ T giây — Dual-trigger pattern.

Giải pháp thực tế: Dual-trigger Buffered Flush

```

1 # batch/object_store_writer.py
2 CONSUMER_FLUSH_SIZE      = int(os.getenv("CONSUMER_FLUSH_SIZE", "500"))
3 CONSUMER_FLUSH_INTERVAL  = int(os.getenv("CONSUMER_FLUSH_INTERVAL", "60"))
4
5 def main_loop(consumer, s3):
6     buffer = defaultdict(list) # key: (platform, year, month, day)
7     last_flush_time = time.time()
8
9     while RUNNING:
10         msg = consumer.poll(timeout=1.0)
11         if msg and not msg.error():
12             row = flatten(json.loads(msg.value().decode("utf-8")))
13             key = partition_key(row)
14             buffer[key].append(row)
15
16         total_buffered = sum(len(v) for v in buffer.values())
17         elapsed = time.time() - last_flush_time
18
19         # Điều kiện flush kép: đủ số lượng HOAC đủ thời gian
20         if total_buffered >= CONSUMER_FLUSH_SIZE or \
21            (elapsed >= CONSUMER_FLUSH_INTERVAL and total_buffered > 0):
22             for key, rows in list(buffer.items()):
23                 _flush_to_minio(s3, key, rows)
24             buffer.clear()
25             last_flush_time = time.time()

```

Listing 35: Dual-trigger Buffered Flush cho Object Store Writer

Cấu trúc phân vùng trên MinIO sau khi áp dụng giải pháp:

```
1 social-lake/
2   data/raw/
3     facebook/2026/01/15/batch_1737000000_abc123.parquet (~5MB)
4     facebook/2026/01/15/batch_1737003600_def456.parquet (~5MB)
5     reddit/2026/01/15/batch_1737000000_ghi789.parquet (~3MB)
6     instagram/2026/01/16/batch_1737090000_jkl012.parquet (~4MB)
```

Listing 36: Cấu trúc thư mục MinIO sau khi áp dụng Dual-trigger Flush

Mỗi file Parquet chứa ~500 bản ghi (thay vì 1 bản ghi), kích thước tăng từ vài KB lên 3–5 MB — trong khoảng tối ưu cho object storage.

Kết quả:

- Số lượng file trong MinIO giảm 98% (từ hàng chục nghìn file xuống vài trăm file).
- Bước list files của Spark Batch Job giảm từ vài phút xuống vài giây.
- Tổng thời gian thực thi Airflow DAG giảm từ ~30 phút (timeout) xuống ~20 phút ổn định.

Bài học kinh nghiệm (Key Takeaways)

- **Object Storage không phải Filesystem:** MinIO và S3 không được thiết kế để có hàng triệu object nhỏ. Mục tiêu thiết kế: file size 64MB–256MB là lý tưởng cho Parquet trên object storage.
- **Dual-trigger flush cân bằng latency và throughput:** Flush theo số lượng (throughput guarantee) kết hợp với flush theo thời gian (latency guarantee) là pattern phổ biến trong data engineering.
- **Partitioning strategy ảnh hưởng trực tiếp đến query performance:** Phân vùng theo platform/year/month/day cho phép Spark bỏ qua (prune) toàn bộ các partition không liên quan khi có date-range filter.

4.6 Bài học 6: Data Deduplication trong Serving Layer — Merge Strategy cho Lambda Architecture**Mô tả vấn đề**

- **Bối cảnh:** Lambda Architecture tồn tại hai nguồn dữ liệu cho cùng một bài đăng: Batch Index (dữ liệu đã tổng hợp từ Spark Job, chứa sentiment score tính bởi lexicon) và Realtime Index (dữ liệu enriched bởi NLP Pipeline thời gian thực, chứa sentiment score tính bởi PhoBERT + keywords + entities).
- **Thách thức:**
 1. **Trùng lặp post_id:** Cùng một bài đăng xuất hiện ở cả hai Elasticsearch index. Dashboard hiển thị bài đăng bị nhân đôi.
 2. **Conflict Resolution:** Hai phiên bản của cùng bài đăng có thể có sentiment_score khác nhau (lexicon vs PhoBERT), keywords chỉ có ở realtime version.

3. **Freshness vs Accuracy:** Realtime version mới hơn nhưng có thể chứa enrichment kém chính xác hơn. Batch version cũ hơn nhưng đã qua làm sạch kỹ lưỡng.
 4. **Timestamp parsing phức tạp:** Dữ liệu từ 3 nền tảng có định dạng timestamp khác nhau (ISO 8601 có timezone, UNIX timestamp, format đặc thù).
- **Tác động hệ thống:** Dashboard hiển thị bài đăng trùng lặp. Biểu đồ Sentiment Trend bị méo vì cùng một bài đăng được tính hai lần trong aggregation.

Các phương pháp tiếp cận

- **Phương pháp 1: Batch overwrites Realtime.** Luôn ưu tiên batch version khi có conflict.
Hạn chế: Bài đăng rất mới chưa có batch view sẽ không xuất hiện trên Dashboard cho đến batch cycle tiếp theo (~5 phút). Vi phạm nguyên tắc realtime của Speed Layer.
- **Phương pháp 2: Realtime overwrites Batch.** Luôn ưu tiên realtime version.
Hạn chế: Mất đi sự chính xác của batch processing. Dữ liệu đã làm sạch bị overwrite bởi dữ liệu realtime có thể chứa lỗi.

Giải pháp thực tế

Nhóm triển khai chiến lược “**Latest event_ts wins**” — luôn giữ version có event_ts mới nhất:

```

1 # serving/merge_service.py
2 @classmethod
3 def _parse_event_ts(cls, post: dict) -> datetime:
4     """Parse event timestamp, xử lý mọi định dạng."""
5     ts = post.get("event_ts") or post.get("created_at")
6     if not ts:
7         return datetime.min.replace(tzinfo=timezone.utc)
8     if isinstance(ts, datetime):
9         return ts if ts.tzinfo else ts.replace(tzinfo=timezone.utc)
10    try:
11        return datetime.fromisoformat(
12            str(ts).replace("Z", "+00:00")
13        ).astimezone(timezone.utc)
14    except Exception:
15        return datetime.min.replace(tzinfo=timezone.utc)
16
17 def _merge_and_dedup(self, posts: list[dict]) -> list[dict]:
18     """Merge danh sách posts, dedup theo post_id."""
19     merged: dict[str, dict] = {}
20     for post in posts:
21         pid = post.get("post_id")
22         if not pid:
23             continue
24         if pid not in merged:
25             merged[pid] = post
26         else:
27             existing_ts = self._parse_event_ts(merged[pid])
28             new_ts = self._parse_event_ts(post)
29             if new_ts > existing_ts:
30                 merged[pid] = post
31             # Neu bang nhau: uu tien version co enrichment data
32             elif new_ts == existing_ts and \

```

```
33         post.get("enrichment") and \
34         not merged[pid].get("enrichment"):
35             merged[pid] = post
36     return list(merged.values())
```

Listing 37: Chiến lược Latest event_ts wins trong Merge Service

Ưu điểm của chiến lược “Latest event_ts wins”:

- Với bài đăng có trong cả batch và realtime: **event_ts** giống nhau (cùng là thời điểm bài đăng được tạo). Khi **event_ts** bằng nhau, ưu tiên version có **enrichment** data (realtime) để tận dụng NLP enrichment.
- Với bài đăng chỉ có trong realtime (chưa được Batch Job xử lý): hiển thị ngay, không phải đợi batch cycle.
- Với bài đăng chỉ có trong batch (quá cũ, đã expire khỏi realtime ES): lấy từ batch, không mất dữ liệu lịch sử.

Kết quả:

- Dashboard không còn hiển thị bài đăng trùng lặp.
- Biểu đồ Sentiment Trend tính toán chính xác hơn (không có double-counting).
- Bài đăng mới xuất hiện trên Dashboard trong vòng 5-10 giây.

Bài học kinh nghiệm (Key Takeaways)

- **Event Time là chân lý, không phải Processing Time:** Merge strategy luôn nên dựa trên **event_ts** (thời điểm sự kiện xảy ra trong thế giới thực), không phải **ingested_at** hay **processed_at**.
- **Timestamp normalization là bước không thể bỏ qua:** Dữ liệu từ nhiều nguồn luôn có định dạng timestamp không đồng nhất. Phải chuẩn hóa về UTC trước khi so sánh.
- **Idempotency trong Merge:** Merge Service nên cho kết quả giống nhau dù gọi bao nhiêu lần với cùng dữ liệu đầu vào.

4.7 Tổng hợp các bài học khác

Table 2: Tổng hợp các bài học kinh nghiệm khác

Category	Key Insight / Best Practice
System Integration	Sử dụng Kubernetes Service Discovery (DNS name) thay vì IP tĩnh cho tất cả service-to-service communication. Ví dụ: <code>kafka-service.social-pipeline.svc.cluster.local:9092</code> thay vì hardcoded IP pod. Pod IP thay đổi sau mỗi lần restart.
Security	Không bao giờ hardcoded credentials (MinIO Key, ClickHouse password) trong mã nguồn. Sử dụng Kubernetes Secrets và mount vào Pod dưới dạng biến môi trường. Thêm validation trong <code>config/settings.py</code> để fail-fast nếu dùng credential mặc định trên Production.
Data Format	Luôn ưu tiên Parquet thay vì JSON/CSV cho dữ liệu lưu trữ dài hạn. Parquet: columnar storage, compression tốt (Snappy), schema tích hợp, hỗ trợ Column Pruning — tất cả đều quan trọng cho Spark Batch Job.
Monitoring	Mọi long-running process (Simulator, Object Store Writer, Speed Streaming) cần expose ít nhất <code>/health</code> endpoint và Prometheus metrics. Không có observability = không thể debug production issues.
DAG Idempotency	Airflow DAG phải idempotent: chạy lại nhiều lần cùng khoảng thời gian phải cho kết quả giống nhau. Cơ chế: kiểm tra MinIO marker trước khi chạy Spark Job, tránh re-process dữ liệu đã xử lý.

References

- [1] J. Lin, “The lambda and the kappa,” *IEEE Internet Computing*, vol. 21, no. 5, pp. 60–66, 2017.
- [2] N. Garg, *Apache Kafka*. Packt Publishing, 2013.
- [3] H. Karau, A. Konwinski, P. Wendell, and M. Zaharia, *Learning Spark: Lightning-Fast Big Data Analysis*. O’Reilly Media, 2015.
- [4] VinAI Research, “PhoBERT: Pre-trained language models for Vietnamese,” *Findings of EMNLP*, 2020.
- [5] T. Akidau et al., “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *PVLDB*, vol. 8, no. 12, pp. 1792–1803, 2015.
- [6] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop distributed file system,” in *Proc. IEEE MSST*, 2010, pp. 1–10.